

Digital Technology

Hoofdstuk 2 – Digitaal rekenen



VIVES University of Applied Sciences
Bachelor in electronics-ICT



— Digital Technology

Hoofdstuk 2: Digitaal rekenen

Inhoud

1. **Getallenstelsels**
2. Omzetting
3. Bewerkingen
4. Vaste en drijvende komma
5. Binair gecodeerde decimale getallen (BCD getallen)

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

1. Getallenstelsels

- Een getal N kan onder de meest algemene vorm geschreven worden als:
- $N = d_{n-1} \cdot r^{n-1} + d_{n-2} \cdot r^{n-2} + \dots + d_0 \cdot r^0 + d_{-1} \cdot r^{-1} + \dots + d_{-m} \cdot r^{-m}$
- $N = \sum_{i=-m}^{n-1} d_i \cdot r^i$ met $0 \leq d_i \leq (r - 1)$
- Met:
 - 'd' een digit, een cijfer
 - 'r' het grondgetal (2, 8, 10, 16)
 - 'n' het aantal cijfers vóór de komma
 - 'm' het aantal cijfers na de komma

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

1. Getallenstelsels

- *Voorbeelden*

- $(123.45)_{10} = 1 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0 + 4 \cdot 10^{-1} + 5 \cdot 10^{-2}$
- $(101.01)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} = (5.25)_{10}$
- $(235)_8 = 2 \cdot 8^2 + 3 \cdot 8^1 + 5 \cdot 8^0 = (157)_{10}$
- $(18D)_{16} = 1 \cdot 16^2 + 8 \cdot 16^1 + D \cdot 16^0 = (397)_{10}$

- d_{n-1} = de eerste digit
- = het meest beduidend cijfer: **MSD** (**M**ost **S**ignificant **D**igit)
- = in het geval van een bit: **MSB** (**M**ost **S**ignificant **B**it)

- d_m = de laatste digit
- = het minst beduidend cijfer: **LSD** (**L**east **S**ignificant **D**igit)
- = in het geval van een bit: **LSB** (**L**east **S**ignificant **B**it)

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

1. Getallenstelsels

- In tabelvorm worden de relaties tussen de verschillende getallenstelsels weergegeven.

Getallenstelsels			
Decimaal	Hexadecimaal	Octaal	Binair
0	0	0	00000
1	1	1	00001
2	2	2	00010
3	3	3	00011
4	4	4	00100
5	5	5	00101
6	6	6	00110
7	7	7	00111
8	8	10	01000
9	9	11	01001
10	A	12	01010
11	B	13	01011
12	C	14	01100
13	D	15	01101
14	E	16	01110
15	F	17	01111
16	10	20	10000

Tabel 2.1: overzicht getallenstelsels

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

1. Getallenstelsels

- Het binair getallenstelsel wordt ook nog wel eens de 8421-code genoemd, immers:

- *Voorbeeld*

- $(1011)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$
- $= 1 \cdot \mathbf{8} + 0 \cdot \mathbf{4} + 1 \cdot \mathbf{2} + 1 \cdot \mathbf{1}$
- $= 8 + 2 + 1 = (11)_{10}$

- $(1111)_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$
- $= 1 \cdot \mathbf{8} + 1 \cdot \mathbf{4} + 1 \cdot \mathbf{2} + 1 \cdot \mathbf{1}$
- $= 8 + 4 + 2 + 1 = (15)_{10}$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

Inhoud

1. Getallenstelsels
- 2. Omzetting**
3. Bewerkingen
4. Vaste en drijvende komma
5. Binair gecodeerde decimale getallen (BCD getallen)

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

2. Omzetting

2.1 Omzetting van binair naar octaal

- Verdeel het binair getal in groepjes van 3 bits, te beginnen met de **LSB**.
- Vervolgens geeft men voor elk groepje het overeenkomstig, octaal getal.

- *Voorbeeld*

- $(\underline{10} \ \underline{100} \ \underline{111} \ \underline{011})_2 = (2473)_8$
- $(\ 2 \quad 4 \quad 7 \quad 3 \)_8$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

2. Omzetting

2.2 Omzetting van octaal naar binair

- De werkwijze is de omgekeerde bewerking van de voorgaande omzetting.
- *Voorbeeld*
- $(3072)_8 = (\underline{011} \ \underline{000} \ \underline{111} \ \underline{010})_2$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

2. Omzetting

2.3 Omzetting van binair naar hexadecimaal

- Verdeel het binair getal in groepjes van 4 bits, beginnend bij de **LSB**.
- Vervolgens geeft men voor elk groepje het overeenkomstig hexadecimaal getal.

- *Voorbeeld*

- $(\underline{110} \ \underline{1101} \ \underline{1001} \ \underline{1110})_2 = (6 \ D \ 9 \ E)_{16}$
- $(\ 6 \ \ 13 \ \ 9 \ \ 14)_{10}$
- $(\ 6 \ \ D \ \ 9 \ \ E)_{16}$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

2. Omzetting

2.4 Omzetting van hexadecimaal naar binair

- De werkwijze is de omgekeerde bewerking van de bovenstaande omzetting.
- *Voorbeeld*
- $(D\ 3\ 9\ A)_{16} = (\underline{1101}\ \underline{0011}\ \underline{1001}\ \underline{1010})_2$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

2.5 Omzetting van decimaal naar binair, octaal of hexadecimaal

- Deel het decimaal getal door de nieuwe basis; men verkrijgt het quotiënt q_1 en onthoud de rest r_1 .
- Deel opnieuw het quotiënt q_1 door de nieuwe basis en onthoud de rest r_2 .
- Herhaal deze bewerking tot het quotiënt = 0.
- Schrijf het resultaat te beginnen met de hoogste restindex: $r_n r_{n-1} r_{n-2} \dots r_2 r_1$
- *Voorbeeld:* $(3817)_{10} = (????)_8$
- $3817 : 8 = 477$ (q_1) met rest $r_1 = 1$
- $477 : 8 = 59$ (q_2) met rest $r_2 = 5$
- $59 : 8 = 7$ (q_3) met rest $r_3 = 3$
- $7 : 8 = 0$ (q_4) met rest $r_4 = 7$
- *Antwoord:* $(3817)_{10} = (7351)_8$
- Wanneer men de conversie uitvoert van een *decimaal* getal naar een *binair* getal kan het gebeuren dat het aantal delingen hoog oploopt.
- Eenvoudiger is het dan om eerst het *decimaal* getal om te zetten naar een *octaal* getal, om dit vervolgens naar een *binair* getal te converteren met de reeds bekende regels.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

2.6 Omzetting van fractionele getallen

- De meest algemene decimale uitdrukking van $(0.35)_{10} = \frac{3}{10^1} + \frac{5}{10^2}$
- Op dezelfde manier is de uitdrukking van $(0.35)_8 = \frac{3}{8^1} + \frac{5}{8^2}$
- $= (0,375)_{10} + (0,078125)_{10}$
- $= (0,453125)_{10}$
- We sluiten af met een binair voorbeeld:
- $(0.111)_2 = \frac{1}{2^1} + \frac{1}{2^2} + \frac{1}{2^3} = (0,5)_{10} + (0,25)_{10} + (0,125)_{10}$
- $= (0,875)_{10}$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

2.7 Omzetting van decimale fracties naar hexadecimale, octale of binaire fracties

- Vermenigvuldig de decimale fractie met de nieuwe basis.
- Het cijfer voor de komma wordt afgezonderd en vormt een cijfer van de nieuwe fractie.
- Herhaal deze bewerking en zonder telkens het cijfer voor de komma af.
- *Voorbeeld*
- $(0.326)_{10} = (????)_8$
- $0.326 \times 8 = 2.608$ zonder het cijfer voor de komma, dus **2**, af
- $0.608 \times 8 = 4.864$ zonder **4** af
- $0.864 \times 8 = 6.912$ zonder **6** af
- $0.912 \times 8 = 7.296$ zonder **7** af
- ...
- Het resultaat heeft geen einde en het systeem treedt beperkend op.
- Het antwoord wordt: $(0.326)_{10} = (0.\mathbf{2467}\dots)_8$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

Inhoud

1. Getallenstelsels
2. Omzetting
- 3. Bewerkingen**
4. Vaste en drijvende komma
5. Binair gecodeerde decimale getallen (BCD getallen)

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.1 Octaal optellen

- Op analoge wijze met de decimale optelling worden de getallen onder elkaar gezet en *rang per rang* (decimaal) opgeteld.
- De som kan nu uit *twee nieuwe rangen* bestaan (omdat de som groter wordt dan 8), waardoor de omzetting van decimaal naar octaal dient te gebeuren: men deelt de som door 8, het quotiënt is het cijfer dat overgedragen moet worden naar de volgende rang.
- De rest blijft behouden op de huidige rang. Ofwel trekt men een aantal keer het getal 8 af van de som. Dit aantal is het over te dragen cijfer.
- De rest blijft eveneens behouden.
- **Voorbeeld:** $(522)_8 + (417)_8 = ?$
- $$\begin{array}{r} 522 \\ + \quad 417 \\ \hline 1141 \end{array}$$

	laagste rang:	$2 + 7 = (9)_{10} = (11)_8$
	middenrang:	$2 + 1 + 1 = (4)_{10} = (4)_8$
	hoogste rang:	$5 + 4 = (9)_{10} = (11)_8$
- Het resultaat $(1141)_8$ bevat meer rangen dan de optellers. Indien het systeem maar beschikt over drie rangen is deze vierde rang verloren.
- Binair noemt men dit een **carry bit**, bij **unsigned arithmetic**.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.2 Hexadecimale optelling

- Rang per rang wordt omgevormd naar het decimale stelstel.
- De decimale rangen worden opgeteld.
- Het resultaat wordt terug omgevormd naar de hexadecimale schrijfwijze. Het aantal keren dat men het getal 16 kan aftrekken van het resultaat is het over te dragen cijfer.
- *Voorbeeld:* $(4A)_{16} + (E5)_{16} = ?$
 - $4A$ laagste rang: $(A)_{16} + (5)_{16} = (10)_{10} + (5)_{10} = (15)_{10} = (F)_{16}$
 - $+ \quad \underline{E5}$ hoogste rang: $(4)_{16} + (E)_{16} = (4)_{10} + (14)_{10} = (18)_{10} = (12)_{16}$
 - $\quad \quad \quad 12F$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.a Binair optellen

- De **optelling** steunt op volgende wetmatigheden:

- $0+0 = 0$
- $1+0 = 1$
- $0+1 = 1$
- $1+1 = \mathbf{1\ 0}$ in dit geval blijft 0 op de desbetreffende rang staan en 1 wordt *overgedragen* (**carry-out**) naar de volgende rang

- *Voorbeeld*

- $$\begin{array}{r} 1\ 1\ 1 \\ +\ 1\ 0\ 1 \\ \hline \end{array}$$
- $$\begin{array}{r} \mathbf{1}\ 1\ 0\ 0 \end{array}$$
- $\underline{1\ 1\ 1}$ overdrachten (carry-out's)
- op de derde rang heeft men: $(1 + 1) + \mathbf{1} = 10 + 1 = \mathbf{1\ 1}$

- Het resultaat vertoont een *carry* bit en kan eventueel verloren zijn voor het systeem, als er gewerkt wordt met een beperkte woordlengte van 3 bits.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.b Het binaire verschil

- Het **verschil** steunt op volgende wetmatigheden:
 - $1-1 = 0$
 - $1-0 = 1$
 - $0-0 = 0$
 - $0-1 = \mathbf{1}$

'1' op deze rang en een '1' *lenen* van de hogere rang, zodat $(1\mathbf{0})_2 - (0\mathbf{1})_2 = (0\mathbf{1})_2$. Vergeet dus niet de geleende bit in mindering te brengen voor de hogere rang!
- Om deze bewerking te kunnen uitvoeren moet men één '1' gaan *lenen* (= to borrow) op de volgende rang.
- Daardoor verhoogt men de huidige rang met $(10)_2$ (ofwel decimaal 2). Dan is $(10)_2 - (1)_2 = (1)_2$.
- Een '1' lenen op een rang betekent een **borrow out** voor deze rang.
- Een '2' krijgen op een rang betekent een **borrow in** voor deze rang.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.b Het binaire verschil

- *Voorbeeld:* $(1100)_2 - (111)_2 = ?$
- De rechtse twee rangen worden uitgewerkt.
- $$\begin{array}{r} \\ \\ \\ \\ \end{array}$$
- $$\begin{array}{r} \\ \\ \\ \\ \end{array}$$
- Op de laagste rang geldt $0-1 = 1$, als een 1 wordt geleend op de vorige rang.
- Daar staat echter eveneens een 0, waardoor er opnieuw een 1 geleend wordt op de rang ervoor.
- De linkse twee rangen worden verder uitgewerkt.
- $$\begin{array}{r} \\ \\ \\ \\ \end{array}$$
- $$\begin{array}{r} \\ \\ \\ \\ \end{array}$$
- Decimaal is $(1100)_2 - (111)_2$ dus $(12)_{10} - (7)_{10} = (5)_{10} = (0101)_2$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.b Het binaire verschil

- *Regel:* om een 1 te lenen gaat men alle 0 links van deze rang veranderen in een 1 en de eerste 1 wordt 0.
- *Voorbeeld:* $(10010011100)_2 - (01110101110)_2 = (?)_2$

- $$\begin{array}{cccccccccc} & 0 & 1 & 1 & 0 & 1 & & 0 & 0 & 0 & \\ & \cancel{1} & \cancel{0} & \cancel{0} & \cancel{1} & \cancel{0} & 0 & \cancel{1} & \cancel{1} & \cancel{1} & 0 & 0 \\ - & 0 & 1 & 1 & 1 & 0 & \underline{1} & 0 & 1 & 1 & \underline{1} & 0 \\ \hline & 0 & 0 & 0 & \color{red}{1} & 1 & \color{green}{1} & 0 & 1 & 1 & \color{green}{1} & 0 \end{array}$$
- met $0 - 1 = 1$, via een borrow

- Een snelle controle valideert het resultaat:
- $(10010011100)_2 - (01110101110)_2 = (1180)_{10} - (942)_{10} = (238)_{10} = (00011101110)_2$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.c De complementaire methode

- Deze methode maakt het mogelijk om na een bepaalde omvorming een **aftrekking** te vervangen door een **optelling**. In de praktijk zal elke bewerking door een **optelling** worden vervangen.
- *Decimaal*
- Voorbeeld: $524 - 273 = 524 + (-273)$
- Het getal -273 wordt **positief** gemaakt door de bewerking $(1000-273)$ uit te voeren, om nadien het resultaat met 1000 te verminderen.
- De bewerking $(1000-273) = 727$ wordt het **tien-complement** van 273 genoemd.
- De uiteindelijke vergelijking is:
 - $524 + (1000-273) - 1000 =$
 - $524 + 727 - 1000 =$
 - $1\mathbf{251} - 1000 = \mathbf{251}$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.c De complementaire methode

- *Voorbeeld:* $(372)_{10} - (124)_{10} = ?$
- $\begin{array}{r} 372 \\ -124 \end{array} \Rightarrow 10\text{-complement} \Rightarrow + \begin{array}{r} 372 \\ \underline{876} \\ 1 \end{array} 248$
- De **carry** wordt weggelaten.
- *Besluit*
- Verkrijgen we in het resultaat een **carry**, dan wijst dit erop dat het resultaat **positief** is.
- In het bovenstaande voorbeeld is het resultaat dus **+248**.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.c De complementaire methode

- *Voorbeeld:* $(273)_{10} - (524)_{10} = ?$
- $$\begin{array}{r} 273 \\ -524 \Rightarrow 10\text{-complement} \Rightarrow + \end{array} \begin{array}{r} 273 \\ \underline{476} \\ 749 \end{array}$$
- Er is in dit voorbeeld *geen carry*!
- *Besluit*
- Indien het resultaat *geen carry* bevat, wijst dit op het feit dat deze uitkomst **negatief** is en geschreven staat in het 10-complement.
- Het negatief 10-complement van 749 is $-(1000-749) = -251$.
- *Opmerking:* uiteraard heeft men nog steeds een aftrekking; men moet immers het getal van een tienvoud aftrekken (in dit geval 10^3) om het 10-complement te berekenen. Ook dit komt te vervallen door de volgende methode: het bepalen van het (Basis-1)-complement!

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.c De complementaire methode

- Het **9-complement** van een getal wordt gevormd door een cijfer per rang, dat bijgevoegd moet worden om per rang 9 te verkrijgen.
- Zo is het 9-complement van 273 het getal 726.
- Het **10-complement** van 273 is 727 of het **9-complement+1**.
- De aftrekking kan men met deze methode als volgt noteren:
- $524 - 273 = 524 + (-273) = 524 + (999 - 273) - 999$
- $= 524 + (999 - 273) + 1 - 1000$
- Van het verkregen resultaat $524 + 726$ wordt 1000 afgetrokken en **nadien 1 bijgeteld**.
- $$\begin{array}{r} 524 \\ - 273 \Rightarrow 9\text{-complement} \Rightarrow + 726 \\ \hline 251 \end{array}$$
- $$\begin{array}{r} 524 \\ - 1000 \\ \hline 250 \end{array}$$
- $$\begin{array}{r} 250 \\ + 1 \\ \hline 251 \end{array}$$
- de 1 vooraan is een carry bit, deze wordt teruggekoppeld en bij het resultaat gevoegd indien er een carry is, duidt dit op een **positief** resultaat.
- Het resultaat is dus **+ 251**, want er is wel een **carry**.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.c De complementaire methode

- *Voorbeeld*
- $$\begin{array}{r} 273 \\ - 524 \\ \hline \end{array} \Rightarrow 9\text{-complement} \Rightarrow \begin{array}{r} 273 \\ + 475 \\ \hline \end{array}$$
- $$\begin{array}{r} 273 \\ - 251 \\ \hline \end{array} \quad 748 \quad \text{geen carry, dus is het resultaat **negatief**}$$
- en staat uitgedrukt in 9-complement
- Het optreden van een **carry** bepaalt dus als het resultaat **positief** of **negatief** is.
- Om het resultaat te bekomen van dit **negatieve** resultaat (er is geen carry!) moet 748 dus omgezet worden in het negatief van het 9-complement.
- Het resultaat is dus $-(9_complement(748))$.
- $748 \Rightarrow -(9\text{-complement}(748)) \Rightarrow -251$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.c De complementaire methode

- *Binaire aftrekking*
- Deze is analoog met de **decimale** aftrekking.
- Decimale basis-complement = **10-complement**; binair wordt dit het **2-complement**
- (Basis-1) complement = **9-complement**; binair wordt dit het **1-complement**
- Aangezien er slechts twee symbolen zijn in het binair stelsel wordt het 1-complement van een getal de inverse schrijfwijze: 1 wordt 0 en een 0 wordt 1.
- Het 2-complement kan worden gevonden door eerst het 1-complement te nemen en dat te verhogen met 1.
- *Regel: om het 2-complement van een binair getal te vinden, beginnen we bij de laagste rangorde met alle nullen en de eerste één te bewaren, om dan alle volgende cijfers te inverteren.*
- **Voorbeeld**
- $(12)_{10} = (001100)_2$
- $\uparrow\uparrow\uparrow$
- 2-complement: $(110100)_2 = (110011)_2 + (000001)_2$
- *Let op:* een nul vooraan verandert in een 1 (positief wordt negatief)!
- Dit heeft zijn betekenis bij het opschuiven van rangen.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.c De complementaire methode

- *Binaire aftrekking: voorbeeld*

- **Het 2-complement:**

- $(26)_{10} = (11010)_2 \Rightarrow$
- $- (12)_{10} = -(01100)_2 \Rightarrow$ 2-complement $\Rightarrow +$ 10100 ; regel toepassen
- 14
- 1 01110
- $\downarrow$ carry valt weg
- $(1110)_2 = (14)_{10}$

- Een **binaire aftrekking** wordt dus vervangen door een *optelling* met het 2-complement, of een *optelling* met het (1-complement+1) (zie voorbeeld hieronder)!

- **Het 1-complement:**

- $(26)_{10} = (11010)_2 \Rightarrow$
- $- (12)_{10} = -(01100)_2 \Rightarrow$ 1-complement $\Rightarrow +$ 10011 ; alle bits inverteren
- 14
- 1 01101
- $\downarrow$ carry valt weg
- + 1
- $01110 = (14)_{10}$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.3.c De complementaire methode

- *Negatieve resultaten*

- **Het 2-complement:**

- $(12)_{10} = (01100)_2 \Rightarrow$ 01100
- - $(\underline{26})_{10} = -(110\mathbf{10})_2 \Rightarrow$ 2-complement $\Rightarrow$ + 00110 ; regel toepassen
- $-(14)_{10}$ 10010
- geen carry

- Het resultaat is dus **negatief** (geen carry) en staat geschreven in het 2-complement en moet omgezet worden in $-[2_complement(result)]$.

- $(10010)_2 \Rightarrow -[(\mathbf{01101})_2 + (00001)_2] \Rightarrow -(01110)_2 = -(14)_{10}$

- **Het 1-complement:**

- $(12)_{10} = (01100)_2 \Rightarrow$ 01100
- - $(\underline{26})_{10} = -(11010)_2 \Rightarrow$ 1-complement $\Rightarrow$ + 00101 ; bits inverteren
- $-(14)_{10}$ 10001
- geen carry

- Het resultaat is dus **negatief** (geen carry) en staat geschreven in het 1-complement.

- Het 1-complement: $(10001)_2 \Rightarrow -(\mathbf{01110})_2 = -(14)_{10}$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

Het binaire verschil, gebruik van 2's complement

- *Gegeven:* $(0)_{10} - (1)_{10} = ?$
- *Opgave:* - bepaal het binaire verschil en beperk de woordlengte tot 4-bits.
- als het resultaat **negatief** is, bepaal dan de **2's complement** waarde.

- *Oplossing:*

	1	0-1=1	0-1=1	0-1=1	10
		0	0	0	0
-		0	0	0	1
		1	1	1	1

- Bemerk dat in elke rang een **borrow** optreedt.
- Decimaal moet het resultaat $(-1)_{10}$ worden. Via een borrow in de hoogste rang wordt volgende bewerking uitgewerkt $(1\ 0000)_2 - (0001)_2 = (16)_{10} - (1)_{10} = (15)_{10}$
- Het resultaat is echter **negatief** omdat de MSB van het resultaat '1' is.
- De 2's complement waarde van het resultaat wordt de one's complement waarde + 1 of $(0000)_2 + (0001)_2 = (0001)_2 = (1)_{10}$
- *Besluit:* in **signed arithmetic** stelt een binair woord van een bepaalde lengte met alle bits op '1' steeds de waarde $(-1)_{10}$ voor. Alle bits op '0', over de volledige lengte, stelt bijgevolg $(0)_{10}$ voor.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.4 Binair vermenigvuldigen

- De volgende stelregels dienen toegepast te worden:

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 1 = 1$

- Voorbeeld*

- $$\begin{array}{r} 111 \\ \times 101 \\ \hline 111 \\ 000 \end{array}$$

één rangorde opschuiven (bijvoegen van één nul op de laagste rang, die echter niet geschreven wordt)

- $$\begin{array}{r} 111 \\ 100011 \end{array}$$

het resultaat bevat 6 rangen. Het systeem moet dit aankunnen! De woordlengte **verdubbelt** dus!

- Het resultaat is dus $(111)_2 \times (101)_2 = (100011)_2$.
- Decimaal** wordt dit: $7 \times 5 = 35 = 2^5 + 2^1 + 2^0 = 32 + 2 + 1$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.5 Binair delen

- Als de deler kan worden afgetrokken van het deeltal, schrijft men een '1' als quotiënt. Daarna wordt de deler naar rechts verschoven (men plaatst eigenlijk een '0' vooraan) en men begint opnieuw.
- Deze bewerking gedraagt zich zo als een aftrekking die in de praktijk vervangen zal worden door een optelling met de methode van het complement.
- Het decimaal delen kan men eveneens door een reeks aftrekkingen vervangen.
- Er zijn drie methodes om de deling binair uit te voeren:
 - de gewone aftrekking uitvoeren met de bekende stelregels;
 - het 2-complement;
 - het 1-complement.
- *Gelieve deze 3 technieken goed te begrijpen en in te oefenen!!!*

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.5 Binair delen

- *Decimaal delen als een reeks aftrekkingen*
- $45 \mid \underline{6}$
- $- \underline{6} \quad \color{red}{1} \Rightarrow 0$
- -2 het resultaat is **negatief**; verschuiven van rang en het quotiënt wordt 0
- $45 \mid \underline{6}$
- $- \underline{06} \quad \color{green}{1}$
- 39 opnieuw wordt op deze rang geprobeerd als het deeltal in het resultaat kan
- $- \underline{06} \quad \color{green}{+1}$: het quotiënt wordt met 1 verhoogd
- 33 opnieuw proberen tot het resultaat **negatief** wordt. Dan was de laatste aftrekking teveel en men verlaagt het quotiënt met 1.
- ... de bewerking (6 aftrekken) gaat steeds verder tot het resultaat negatief is
- In dit voorbeeld kan men 7 maal 6 aftrekken van 45. De rest is 3, want $(7 \times 6) + 3 = 45$
- Door de kennis van de 'tafels van vermenigvuldiging' weten we uiteraard dat het dichtste geheel veelvoud van 6 om 45 te benaderen 7 is. Helaas kennen we dit binair niet!
- Het **deeltal** = (deler x quotiënt) + rest of $45 = (6 \times 7) + 3$
- **Het quotiënt bepaalt hoeveel maal de deler in het deeltal past!**

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.5 Binair delen

- De gewone aftrekking
- $(45)_{10} = (101101)_2$ en $(6)_{10} = (110)_2$
- $$\begin{array}{r} 101101 \quad | \underline{110} \\ - \underline{110} \qquad \color{red}{\cancel{1}} \Rightarrow 0 \end{array}$$
- Hier moeten we lenen, waardoor het resultaat **negatief** is.
- Het quotiënt wordt met 1 **verlaagd** en er wordt een rang opgeschoven.
- $$\begin{array}{r} 101101 \quad | \underline{110} \\ - \color{red}{0}\underline{110} \qquad \color{red}{1} \\ \hline 0101 \end{array}$$

nu moet er gecontroleerd worden of 110 nogmaals in 0101 kan
Dit is niet het geval zodat er opnieuw een rang wordt opgeschoven.

$$\begin{array}{r} 01010 \\ - \color{red}{00}\underline{110} \quad +1 = (11)_2 = 3 \\ \hline 001001 \end{array}$$
- $$\begin{array}{r} 001001 \\ - \color{red}{000}\underline{110} \quad +1 = (111)_2 = 7 \\ \hline 000011 \end{array}$$

dit is de **rest**
- Het **quotiënt** is: $\color{green}{1} \color{blue}{1} \color{red}{1} = (111)_2 = (7)_{10}$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.5 Binair delen

- De 2-complement methode voor de berekening van $45/6$

- De deler wordt geschreven in het 2-complement: $-(110)_2 \Rightarrow [(001)_2 + (001)_2] = (010)_2$

- $$\begin{array}{r} 101101 \mid \underline{010} \\ + \quad \underline{010} \quad \quad \quad \textcolor{red}{+} \quad \Rightarrow \quad 0 \\ \hline 111 \end{array}$$

geen carry; het resultaat is dus **negatief** waardoor men een rang moet opschuiven. Het quotiënt wordt teruggebracht op 0.

- $$\begin{array}{r} 101101 \mid \underline{010} \\ + \quad \textcolor{red}{1010} \quad \quad \quad \textcolor{red}{1} \\ \hline 10101 \end{array}$$

één rang opschuiven betekent nu een '1' invoegen op de **MSB**

één carry: het resultaat is **positief** en de carry valt weg. Er moet gecontroleerd worden of 010 nogmaals in het resultaat

- $$\begin{array}{r} 0101\textcolor{red}{0} \\ + \quad \textcolor{red}{11010} \quad \quad \quad \textcolor{blue}{11} \\ \hline 100100 \end{array}$$

0101 kan. Dit is weerom niet het geval en er wordt een rang opgeschoven.

één carry: idem.

- $$\begin{array}{r} 00100\textcolor{red}{1} \\ + \quad \textcolor{red}{111010} \quad \quad \quad \textcolor{blue}{111} \\ \hline 10000\textcolor{blue}{11} \end{array}$$

één carry: de rest wordt 0000 $\textcolor{blue}{11}$ of $(11)_2 = (3)_{10}$
het quotiënt is $1 + 1 + 1 = (111)_2$ of $(7)_{10}$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.5 Binair delen

- De 1-complement methode voor de berekening van $45/6$
- De deler wordt in het 1-complement geschreven als $(001)_2$.

$$\begin{array}{r} 101101 \mid \underline{001} \\ + \quad \underline{001} \\ \hline 110 \end{array}$$

$$\begin{array}{r} 101101 \mid \underline{001} \\ + \quad \underline{1001} \quad 1 \\ \hline 10100 \end{array}$$

• 110 het resultaat is **negatief**:
• opschuiven!

$$\begin{array}{r} 10100 \\ \downarrow + 1 \\ 01010 \end{array}$$

$$\begin{array}{r} 01010 \\ + \quad \underline{11001} \quad 11 \\ \hline 100011 \end{array}$$

$$\begin{array}{r} 100011 \\ \downarrow + 1 \\ 001001 \end{array}$$

$$\begin{array}{r} 001001 \\ + \quad \underline{111001} \quad 111 \\ \hline 1000010 \end{array}$$

$$\begin{array}{r} 1000010 \\ \downarrow + 1 \\ 000011 \end{array}$$

• 000011 de rest

- Het quotiënt is: $1 + 1 + 1 = (111)_2 = (7)_{10}$, de rest is $(11)_2 = (3)_{10}$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.6 Voorstelling van positieve en negatieve getallen

- De voorstelling van een **positief** of een **negatief** getal gebeurt met een tekenbit (sign bit).
- Bij conventie wordt deze bit **links** van het getal geplaatst (= **MSB**), met de volgende afspraak:
 - **0** : duidt op een **positief** getal
 - **1** : duidt op een **negatief** getal
- Er zijn nu drie mogelijkheden om een getal weer te geven.
 - Een tekenbit, gevolgd door de **absolute waarde** van het getal:
 - $(+13)_{10} = 0.1101$ en $(-13)_{10} = 1.1101$
 - Een tekenbit, gevolgd door het **2-complement** van het getal:
 - $(+13)_{10} = 0.1101$ en $(-13)_{10} = 1.0011$
 - Een tekenbit, gevolgd door het **1-complement** van het getal:
 - $(+13)_{10} = 0.1101$ en $(-13)_{10} = 1.0010$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.6 Voorstelling van positieve en negatieve getallen

- In tabelvorm wordt dit:

decimaal	absolute waarde	2-complement	1-complement
+5	0 . 0 1 0 1	0 . 0 1 0 1	0 . 0 1 0 1
+4	0 . 0 1 0 0	0 . 0 1 0 0	0 . 0 1 0 0
+3	0 . 0 0 1 1	0 . 0 0 1 1	0 . 0 0 1 1
+2	0 . 0 0 1 0	0 . 0 0 1 0	0 . 0 0 1 0
+1	0 . 0 0 0 1	0 . 0 0 0 1	0 . 0 0 0 1
0	0 . 0 0 0 0	0 . 0 0 0 0	0 . 0 0 0 0
-0	1 . 0 0 0 0	-	1 . 1 1 1 1
-1	1 . 0 0 0 1	1 . 1 1 1 1	1 . 1 1 1 0
-2	1 . 0 0 1 0	1 . 1 1 1 0	1 . 1 1 0 1
-3	1 . 0 0 1 1	1 . 1 1 0 1	1 . 1 1 0 0
-4	1 . 0 1 0 0	1 . 1 1 0 0	1 . 1 0 1 1
-5	1 . 0 1 0 1	1 . 1 0 1 1	1 . 1 0 1 0

Tabel 2.2: voorstelling positieve en negatieve getallen

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.6 Voorstelling van positieve en negatieve getallen

- Bewerkingen met een tekenbit kunnen voor problemen zorgen.
- Voorbeelden in het 2-complement waarbij het resultaat **in orde** is:
- $+6 \Rightarrow 0.0110$ $-6 \Rightarrow 1.1010$
- $\underline{+4} \Rightarrow \underline{0.0100}$ $\underline{+4} \Rightarrow \underline{0.0100}$
- $+10 = 0.1010$ $-2 = 1.1110$
- ↓ **negatief** getal, 2-complement
- $+6 \Rightarrow 0.0110$ $-6 \Rightarrow 1.1010$
- $\underline{-4} \Rightarrow \underline{1.1100}$ $\underline{-4} \Rightarrow \underline{1.1100}$
- $+2 \Rightarrow 1.0.0010$ $-10 \Rightarrow 1.0110$
- ↓
- carry valt weg carry valt weg en 1.0110 is
- het resultaat is **positief** een **negatief** getal, 2-complement

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.6 Voorstelling van positieve en negatieve getallen

- Voorbeelden in het 2-complement waarbij het resultaat **foutief** is:

•	$+ 9$	$\Rightarrow$	0.1001		$- 9$	$\Rightarrow$	1.0111
•	$+ 10$	$\Rightarrow$	0.1010		$- 10$	$\Rightarrow$	1.0110
•	$+ 19$	$=$	1.0011		$- 19$	$=$	10.1101

- het resultaat is **negatief!** Carry valt weg, een **positief** resultaat!
- Op de volgende slide wordt de verklaring voor dit fenomeen besproken!*

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

3. Bewerkingen

3.6 Voorstelling van positieve en negatieve getallen

- Deze verkeerde resultaten treden op omdat:
 - Er een overdracht (carry) gegenereerd werd bij de **optelling** van twee **positieve** getallen op de rang van de **MSB**. Dit resulteerde in een foutieve tekenbit.
 - Er geen overdracht was op de MSB-rang bij de **optelling** van twee **negatieve** getallen, maar wel op de rangorde van de tekenbit waardoor deze overflow verdween en een verkeerde aanduiding ontstond.
- Dit zijn de 'overflow-conditions' of overloop voorwaarden.
- Een voldoende capaciteit kan dit probleem verhelpen. Indien we in voorgaande toepassingen zes bitposities hadden voorzien in plaats van vijf, zou er geen overdracht in de verkeerde rangen worden gegenereerd.
- $+9 \Rightarrow 0.01001$ (er is één bit meer gebruikt)
- $+10 \Rightarrow 0.01010$
- $+19 \Rightarrow 0.10011$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

Inhoud

1. Getallenstelsels
2. Omzetting
3. Bewerkingen
- 4. Vaste en drijvende komma**
5. Binair gecodeerde decimale getallen (BCD getallen)

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

4. Vaste en drijvende komma

4.1 Vaste komma

- In het binair stelsel zijn '0' en '1' de enige symbolen; de tekenbit zorgde reeds voor een extra bit. De komma zou nogmaals een extra bit vragen.
- Om het aantal bits te beperken wordt er een afspraak gemaakt over de plaats van de komma in het getal.
- De komma stelt ons in staat de plaats van de getallen onderling te bepalen:

- $$\begin{array}{r} 568,64 \\ + 12,985 \\ \hline \end{array}$$

- In de fixed-point-notatie (vaste komma) staat de komma echter steeds **links** van alle getalbits. Alle getallen bevinden zich dan tussen 0 en 1.
- Fixed-point betekent immers de vergrendeling van de komma op een **vaste plaats**.

- *Voorbeeld*

- $$\begin{array}{r} 10010,11 \\ + \quad 110,01 \\ \hline \end{array}$$
- $$\begin{array}{r} 11 \\ + 01 \\ \hline \end{array}$$
- $$\begin{array}{r} ?? \\ + ?? \\ \hline \end{array}$$

- Dit wordt: ,10010110 + ,11001000 indien we beschikken over 8 bits.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

4. Vaste en drijvende komma

4.1 Vaste komma

- Op deze wijze de vorige bewerking uitvoeren zou een verkeerd resultaat opleveren, omdat het tweede getal over twee plaatsen moet opschuiven naar rechts.
- Men brengt de komma's onder elkaar:
- •
•
$$\begin{array}{r} ,1\ 0\ 0\ 1\ 0\ 1\ 1\ 0 \\ ,0\ 0\ 1\ 1\ 0\ 0\ 1\ 0 \end{array}$$
 dit wordt **scaling** of 'het op schaal brengen' genoemd.
- De binaire getallen kunnen dus niet zomaar opgeteld worden. De voorafgaande bewerking van **scaling**, voor de operatie, is hier essentieel.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

4. Vaste en drijvende komma

4.2 Drijvende komma

- In deze 'floating point'-notatie wordt een getal voorgesteld door twee groepen bits.
- De *eerste* groep stelt het **getal** zelf voor, de *tweede* groep de plaats van de **komma**.
- Decimaal kan men de getallen als volgt weergeven:
 - $+ 0,00234 = 0,234 \cdot 10^{-2}$
 - $- 52,36 = -0,5236 \cdot 10^2$
- Indien de exponent **negatief** is, moet de komma **naar links** worden opgeschoven.
- Indien de exponent **positief** is, moet de komma **naar rechts** worden geschoven, in beide gevallen een aantal rangen dat bepaald wordt door de waarde van de exponent.
- Binair wordt dit:
 - $(1101011)_2 = 0,1101011 \cdot 2^7$

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

4. Vaste en drijvende komma

4.2 Drijvende komma

- Omdat er geen exponenten weergegeven kunnen worden, plaatst men de waarde van de exponent na het getal.
- Men werkt in het binair stelsel, waardoor er geen verwarring mogelijk is.
- Vervolgens wordt de notatie **0,** eveneens weggelaten.

- *Voorbeelden:*

<u>oorspronkelijk getal</u>	=	<u>mantisse</u>	<u>exponent</u>
+ 11,01101	=	0, + 1101101	+ 010
- 1101,100	=	0, - 1110100	+ 100
+ 0,000110	=	0, + 1100000	- 011
- 0,010101	=	0, - 1010100	- 001

- Het eerste gedeelte wordt de **mantisse** genoemd, het tweede gedeelte de **exponent**.
- Om de exponent tekenbit te weren wordt er $(1000)_2$ bij geteld.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

4. Vaste en drijvende komma

4.2 Drijvende komma

- Grote getallen kunnen op deze wijze compact worden voorgesteld en ten opzichte van de vaste-komma-notatie is dit een groot voordeel.
- Men kan eveneens de getallen op schaal brengen: de exponenten moeten dan gelijkgemaakt worden.
- Naar rechts schuiven betekent de exponent met één verhogen. Naar links schuiven betekent de exponent met één verlagen.
- *Voorbeeld:* + 110101,0 en + 1001,100
- Floating point: + 1101010 + 110 op schaal brengen: + 1101010 + 110
- + 1001100 + 100 + 0010011 + 110
- + 1111101 + 110
- In deze notatie wordt de optelling dan alleen uitgevoerd op de mantisse, de exponent is immers dezelfde.
- Het verschuiven van het resultaat opdat de mantisse zou liggen tussen 0,5 (of $\frac{1}{2}$) en 1 wordt het **normaliseren** genoemd.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

4. Vaste en drijvende komma

4.2 Drijvende komma

- *Opmerking:* $(0,1)_2 = \frac{1}{2} = (0,5)_{10}$
- *Voorbeeld:* + 0000010 + 100 wordt genormaliseerd tot + 1000000 - 001
- De vermenigvuldiging met floating-point-notaties is eenvoudig geworden:
 - De exponenten worden opgeteld
 - De mantissen worden vermenigvuldigd
- *Voorbeeld:* indien men beschikt over zes bits om de mantisse weer te geven en over drie bits voor de exponent wordt de volgende bewerking:

•	110,110	⇒	0,110110	· 2 ⁺⁰¹¹	⇒	110110	+ 011
•	x 10,0100	⇒	x	<u>0,100100</u> · 2 ⁺⁰¹⁰	⇒ x	<u>100100</u>	+ <u>010</u>
•				0,011110011000		011110...	+ 101
•	<i>Normeren:</i>					111100	+ 100

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

Inhoud

1. Getallenstelsels
2. Omzetting
3. Bewerkingen
4. Vaste en drijvende komma
5. **Binaire gecodeerde decimale getallen (BCD getallen)**

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

5. Binair gecodeerde decimale getallen (BCD getallen)

- Een *decimaal* getal kan worden omgezet in een *binair* getal. Indien men echter elk cijfer waaruit een decimaal getal is opgebouwd omzet naar zijn binair equivalent spreekt men van een binair gecodeerd decimaal getal of het **BCD-getal**.
- Met n bits kan men 2^n verschillende combinaties voorstellen. Om 10 cijfers voor te stellen moet men dus beschikken over minstens 4 bits: $2^4 > 10$.
- *Voorbeeld*: $(845)_{10} = (1000\ 0100\ 0101)_{BCD}$
- Met 4 bits kan men 16 combinaties opbouwen.
- Er zijn dan ook verschillende mogelijkheden, waarvan de keuze afhangt van:
 - De methode die gebruikt wordt bij het optellen en aftrekken;
 - De noodzaak om te tellen en te complementeren;
 - De conversies van externe code naar BCD-code;
 - De mogelijkheid om fouten te ontdekken en te verbeteren.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

5. Binair gecodeerde decimale getallen (BCD getallen)

5.1 4 bit codes

- Om de 10 symbolen uit het decimaal stelsel met een 4-bit systeem weer te geven, zijn er $\frac{16!}{6!}$ mogelijkheden. Deze codes zijn niet allemaal verschillend van elkaar.
- Het onderling wijzigen van de nullen en de enen reduceert dit aantal mogelijkheden met een factor 384.
- Er blijven nog steeds $7,6 \cdot 10^7$ mogelijkheden over.
- De meest bekende codes worden in tabelvorm weergegeven.
- De 8421-code, de 2421-, 5211- en de $84\bar{2}\bar{1}$ code zijn gewogen codes.
- Zo'n gewogen code kan als volgt genoteerd worden:
- *Voorbeeld:* de 8421-code: $d = 8.w + 4.x + 2.y + 1.z$
 - Met d: het decimale cijfer
 - Met w,x,y en z: de waarde van de bit (0 of 1) van het codegetal.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

5. Binair gecodeerde decimale getallen (BCD getallen)

5.1 4 bit codes

- Onderstaande tabel geeft een overzicht van de 8421-, de 2421-, de 5211-, de $84\overline{21}$ - en de XS3 codes:

decimaal getal	8 4 2 1	2 4 2 1	5 2 1 1	$8 4 \overline{2} \overline{1}$	XS 3
0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 0 0 1	0 0 0 1	0 1 1 1	0 1 0 0
2	0 0 1 0	0 0 1 0	0 0 1 1	0 1 1 0	0 1 0 1
3	0 0 1 1	0 0 1 1	0 1 0 1	0 1 0 1	0 1 1 0
4	0 1 0 0	0 1 0 0	0 1 1 1	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 1 1	1 0 0 0	1 0 1 1	1 0 0 0
6	0 1 1 0	1 1 0 0	1 0 0 1	1 0 1 0	1 0 0 1
7	0 1 1 1	1 1 0 1	1 0 1 1	1 0 0 1	1 0 1 0
8	1 0 0 0	1 1 1 0	1 1 0 1	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 1 1	1 1 1 1	1 1 1 1	1 1 0 0

Tabel 2.3: overzicht binaire codes

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

5. Binair gecodeerde decimale getallen (BCD getallen)

5.1 4 bit codes

- De laatste code, de **XS3-code** (excess-3) is geen gewogen code volgens de geijkte uitdrukking. De relatie tussen het decimaal getal en de code is als volgt: $d = 8.w + 4.x + 2.y + z - 3$
- Vandaar de benaming 'excess-3' of '3-teveel' code. Om het getal 4 te coderen nemen we in feite de decimale 7 en coderen deze laatste.
- Het voordeel van zulke code is dat er een duidelijk verschil is tussen het getal 0 en het geval waarbij dat er geen informatie op de databus staat. Zo kan men een fysische verbreking controleren. Indien men de code 000 detecteert is er duidelijk een onderbreking, want het getal 0 wordt met de code 0011 weergegeven.
- In een gewogen code krijgt elk van de 4 bits een waarde, zodat de som van de waarden gelijk is aan het overeenkomstig decimaal getal. Het gewicht van een code kan positief of negatief zijn. Indien er één of meer gewichten negatief zijn, spreekt men van een negatief gewogen code.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

5. Binair gecodeerde decimale getallen (BCD getallen)

5.1 4 bit codes

- De **AIKEN-code** of de **2421-code** heeft de eigenschap dat de **MSB=0** voor de cijfers 0 tot en met 4 en dat de **MSB=1** voor de cijfers 5 tot en met 9. Dit biedt voordelen bij het afronden van getallen.
- Een tweede eigenschap is de volgende: indien men het 1-complement neemt van de code, verkrijgt men het **9-complement** van het decimaal getal.
- *Voorbeeld*
- $(3)_{10} = (0\ 0\ 1\ 1)_2$
 - $\downarrow\ \downarrow\ \downarrow\ \downarrow$
 - $(1\ 1\ 0\ 0)_2 = (6)_{10}$ of het 9-complement van 3.
- Deze code wordt ook wel een **zelf-complementerende code** genoemd.

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

5. Binair gecodeerde decimale getallen (BCD getallen)

5.2 Meer dan 4 bit codes

- Omwille van foutverbeterende systemen worden codes gebruikt die meer dan 4 bits vertonen.
- Onderstaande tabel geeft enkele voorbeelden weer:

Decimaal	86421	51111	5043210	2 out of 5	Johnson	Ringteller
0	00000	00000	0100001	00011	00000	0000000001
1	00001	00001	0100010	00101	00001	0000000010
2	00010	00011	0100100	00110	00011	0000000100
3	00011	00111	0101000	01001	00111	0000001000
4	00100	01111	0110000	01010	01111	0000010000
5	00101	10000	1000001	01100	11111	0000100000
6	01000	11000	1000010	10001	11110	0001000000
7	01001	11100	1000100	10010	11100	0010000000
8	10000	11110	1001000	10100	11000	0100000000
9	10001	11111	1010000	11000	10000	1000000000

Tabel 2.4: overzicht meer dan 4 bit codes

— Digital Technology

Hoofdstuk 2: Digitaal rekenen

5. Binair gecodeerde decimale getallen (BCD getallen)

5.2 Meer dan 4 bit codes

- De **51111 code** is een gewogen en een zelf-complementerende code.
- De **2-out-of-5 code** is een *cyclische* code of een 'walking' code. Bij overgang naar een hoger cijfer maken de 'enen' een soort stapbeweging.
- De **Johnson-code** of de schuifregister-code is een niet gewogen code, maar kan gemakkelijk met elektronische schakelingen worden opgebouwd. Immers de linkerbit wordt geïnverteerd en teruggekoppeld naar rechts om ingeschoven te worden (zie later).
- De **ringteller** is erg eenvoudig van opbouw en gemakkelijk te decoderen, omdat er maar één '1' per cijfer is: om te verhogen hoeft men slechts een rotatie van één '1' naar links uit te voeren.

Digital Technology

Einde hoofdstuk 2 – Digitaal rekenen



VIVES University of Applied Sciences
Bachelor in electronics-ICT

